

BioRxiv Search documentación

Descripcion

BioRxiv Search es un motor de búsqueda de artículos científicos de Covid19 utilizando la base de datos Elasticsearch y el repositorio de descripciones de artículos científicos BioRxiv.

Diagramas

A continuación, se presenta el diagrama de flujo y el diagrama de Arquitectura de BioRxiv Search.

Diagrama de Flujo

El siguiente diagrama describe el proceso realizado por el BioRxiv Search.

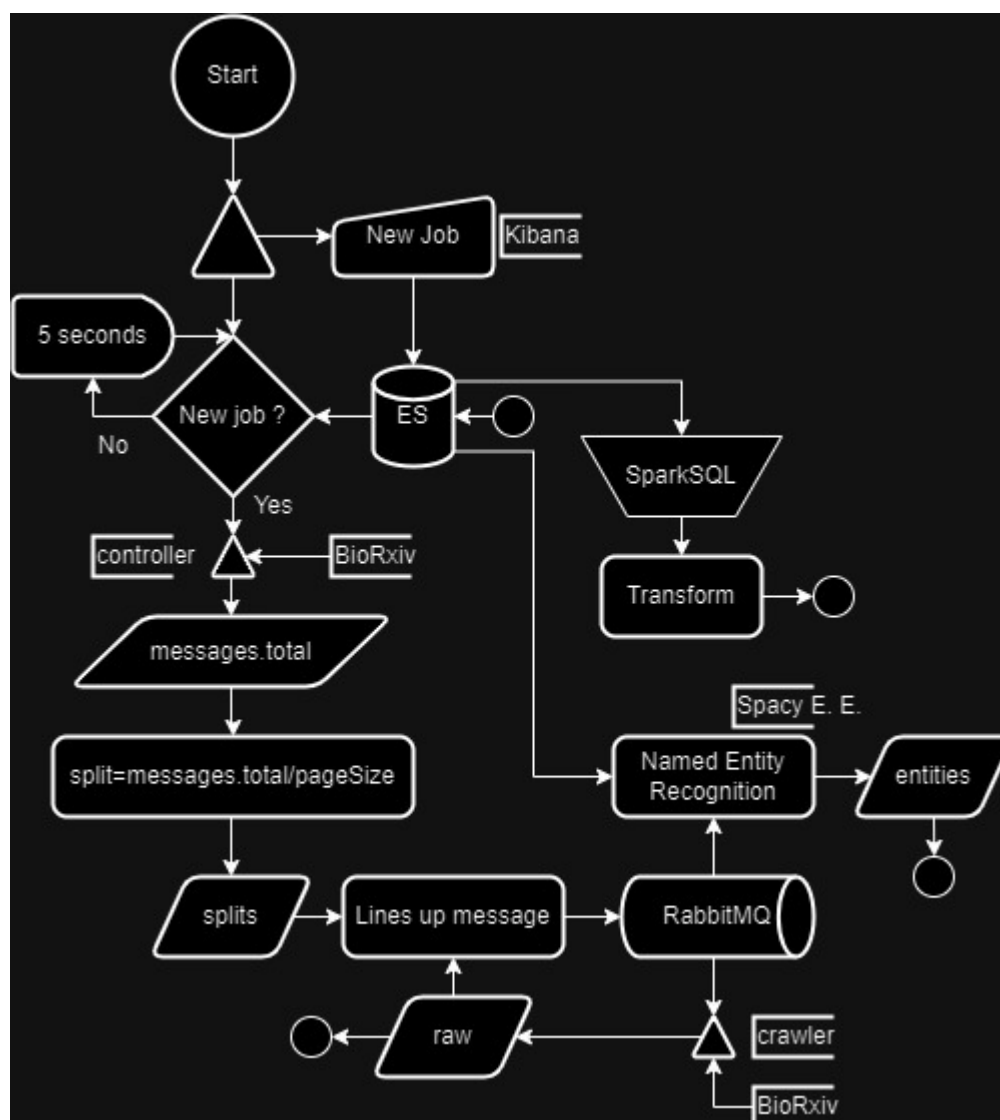


Diagrama de Arquitectura

El siguiente diagrama muestra la implementación e interacción de los componentes del BioRxiv Search. Así como el flujo de datos dentro del sistema.

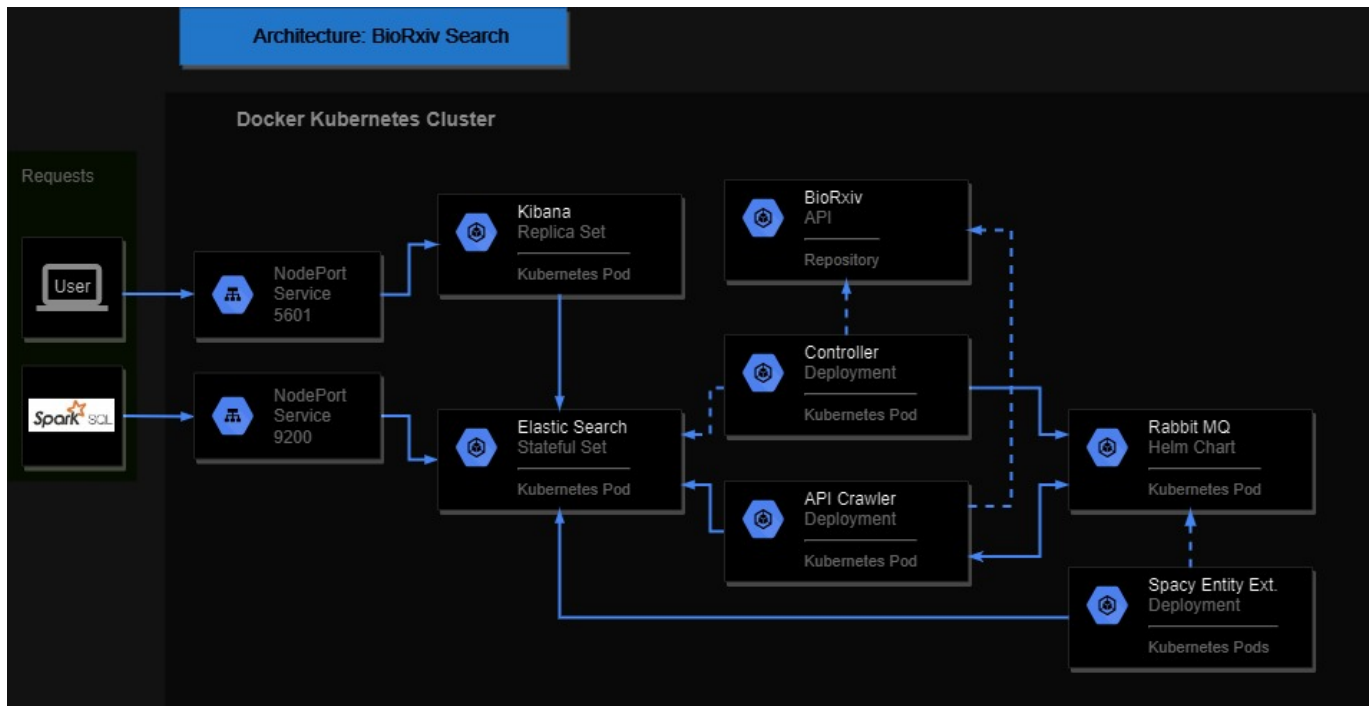
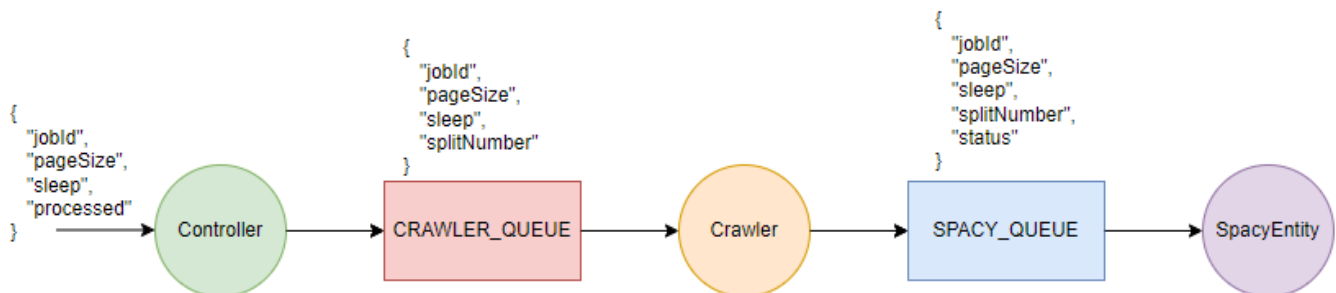


Diagrama de RabbitMQ

El siguiente diagrama muestra cómo están estructuradas las colas de RabbitMQ y la forma en que se comunican los componentes.



Pre-instalación

Como pre-instalación del proyecto serán necesarias las siguientes aplicaciones:

- [Git](#)
- [Python](#)
- [Visual Studio Code](#)
- [Docker](#)
- [Helm](#)
- [Lens](#)
- [SparkSQL](#)
- [Elastic Hadoop](#)

Extraer los elementos en una carpeta definida y copiar `elasticsearch-hadoop-8.6.2.jar` dentro de `spark-2.4.8-bin-hadoop2.7/jars/`. Se debe verificar que el sistema esté corriendo la versión de JDK 1.8 ya que de otra forma no será posible correr el código de scala. Para verificar esto se puede correr el comando de: `java -`

version y verificar que la version de este sea la 1.8. De no ser la versión 1.8 se recomienda para evitar conflictos desinstalar java e instalar la versión necesaria para el proyecto.

Kubernetes será implementado mediante la habilitación del cluster en la aplicación de Docker Desktop.

Instalación

Para la instalación del cluster de Kubernetes es necesario ejecutar los siguientes comandos en el Git Bash:

- `helm repo add elastic https://helm.elastic.com`
- `helm repo add bitnami https://charts.bitnami.com`
- `helm repo update`
- `cd bootstrap`
- `rm -rf Char.lock`
- `helm dependency build --skip-refresh`
- `cd ../stateful`
- `rm -rf Char.lock`
- `helm dependency build --skip-refresh`
- `cd ..`

Estas líneas de comando lo que realizan es importar los repositorios de Bitnami y Elasticsearch que son necesarios para la instalación de sus debidos componentes e instalar las dependencias de cada Helm chart. En seguida se continuará con la instalación de los componentes, empezando con el bootstrap, el cual instala el elastic operator:

- `helm upgrade --install bootstrap bootstrap`

Una vez se instala el bootstrap, se debe ejecutar el siguiente comando para instalar el stateful, el cual consiste en kibana, elasticsearch y rabbitMQ:

- `helm upgrade --install stateful stateful`

Una vez se finaliza de instalar el stateful (se puede verificar con `kubectl get pods` y verificando que todos los pods estén listos, o se puede verificar mediante la aplicación de Lens) se procede a instalar el stateless el cual corresponde al controller, API crawler y el Spacy.

- `helm upgrade --install stateless stateless`

Una vez finalizada la instalación se pueden realizar las pruebas de funcionamiento definidas en la sección de Ejecución y Pruebas Realizadas.

Cuando ya es utilizó la aplicación y se desea desinstalarla, se debe realizar lo siguiente:

1. Desinstalar todos los componentes:

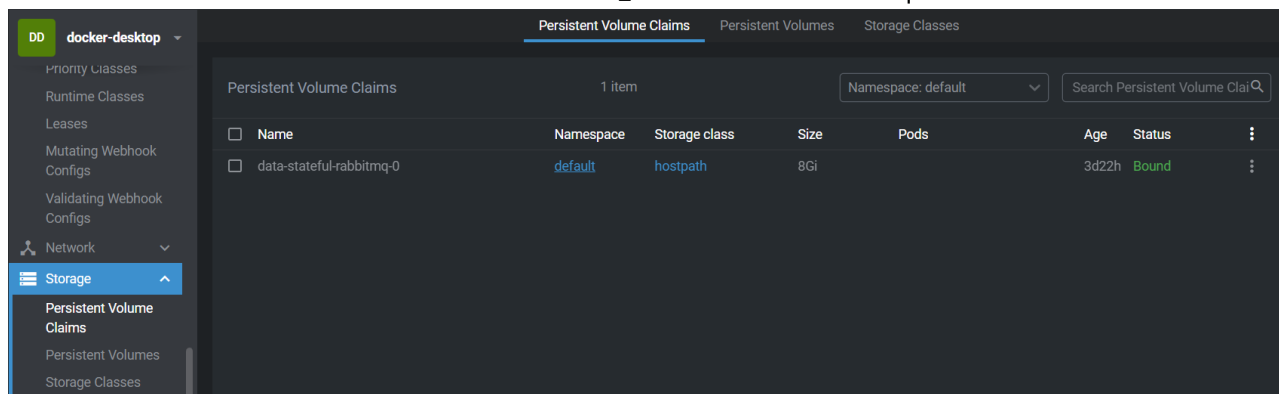
- `helm uninstall stateless stateful bootstrap`

2. Eliminar el Persistent Volume Claim de RabbitMQ:

2.1 En Lens, vamos a *Storage*.

2.2 Ingresamos a la sección de Persistent Volume Claim.

2.3 Borrarnos el Persistent Volume Claim llamado _data-stateful-rabbitmq-0



Descripción de Componentes

Elasticsearch

[Elasticsearch](#) es un motor de búsqueda y análisis de datos. Está diseñado para almacenar, indexar y buscar grandes volúmenes de datos en tiempo real. Utiliza una estructura basada en JSON. Esta es la base de datos principal donde se guardarán los datos extraídos del API de BioRxiv.

Implementación Elasticsearch

Elasticsearch fue implementado mediante el uso de Helm charts, en la dirección: [template elasticsearch](#), se encuentra el template de creación de Elasticsearch, y fuera de esta carpeta se encuentra el archivo de [Values.yaml](#) donde se especifica los valores que debe tomar el template. Es importante notar que dentro del [elastic.yaml](#) se encuentra definido el stateful set de elasticsearch y el servicio tipo nodeport que expone el puerto del pod (9200) a la máquina (localhost) en el puerto 30090.

Kibana

[Kibana](#), por otro lado, es una plataforma de visualización y análisis de datos, esta permite crear visualizaciones interactivas y paneles de control para explorar y analizar los datos almacenados en Elasticsearch. Este será utilizado para la interacción con Elasticsearch por parte de los usuarios.

Implementación Kibana

Elasticsearch fue implementado mediante el uso de Helm Charts, en la dirección: [template kibana](#), se encuentra el template de creación de Kibana, y también se encuentra el archivo de [Values.yaml](#) donde se especifica los valores que debe tomar el template. Es importante notar que dentro del [kibana.yaml](#) se encuentra definido el replica set de kibana y el servicio tipo nodeport que expone el puerto del pod (5601) a la máquina (localhost) en el puerto 30091, por lo que para acceder a kibana es necesario ingresar a localhost:30091. Es importante entrar a la dirección con https: https://localhost:30091

RabbitMQ

[RabbitMQ](#) es un software de intermediación de mensajes que se utiliza para gestionar colas de mensajes entre diferentes aplicaciones o componentes de software. Este será utilizado para enviar mensajes entre los componentes para que realicen sus tareas definidas.

Implementación RabbitMQ

RabbitMQ fue implementado como una dependencia en el archivo de [Charts.yaml](#), la instalación de este es de manera "default" ya que no se especifican valores para sus parámetros. Este pod permanece activo durante la ejecución y son los diferentes componentes los que crean y consumen las colas (esto se verá mas a fondo en la descripción de cada componente).

Spacy Entity Extractor

El Spacy Entity Extractor tiene la funcionalidad de leer los mensajes publicados por el componente API Crawler en una cola de RabbitMQ. Posteriormente, realiza un Named Entity Recognition, donde estas entidades se almacenarán en un nuevo campo de tipo Array llamado entities, en un índice llamado augmented.

Implementación Spacy Entity Extractor

A continuación, se muestra al implementación del componente Spacy Entity Extractor.

```

9  def callback(ch, method, properties, body):
10     json_object = json.loads(body)
11     print(f"Received {json_object}")
12     jobId = json_object["jobId"]
13     splitNumber = int(json_object["splitNumber"])
14
15     documentId = jobId + str(splitNumber)
16     query = {
17         "bool": {
18             "filter":
19                 { "term": { "splitId": documentId }}
20         }
21     }
22     response = es.search(index=ESINDEX, query=query, size=100)
23     print("DocumentId:", documentId, "Response:", response)
24
25     for hit in response["hits"]["hits"]:
26         source_dict = hit["_source"]
27         print("Unprocessed:", source_dict)
28         articles = source_dict.get("articles")
29         for article in articles:
30             # Procesar y agregar las entidades
31             process_and_augment_document(article)
32             print("Processed:", source_dict)
33             # Actualizar el documento en Elasticsearch con el campo "augmented"
34             es.index(index=AUGMENTEDINDEX, id=hit["_id"], document=json.dumps(source_dict, default=set_encoder))
35             print(f"Processed and updated document {hit['_id']}\n")
36     time.sleep(int(json_object["sleep"])/1000)
37

```

En la figura anterior se muestra el método callback, encargado de leer los mensajes publicados por el API Crawler en la cola de RabbitMQ. Este componente está recibiendo mensajes de la cola extraída de la variable de entorno SPACY_QUEUE.

```

38 def set_encoder(obj):
39     if isinstance(obj, set):
40         return list(obj)
41     raise TypeError(f"Object of type {type(obj).__name__} is not JSON serializable")
42
43 def perform_ner(text):
44     doc = nlp(text)
45     entities = [ent.text for ent in doc.ents]
46     return entities
47
48 def process_and_augment_document(article):
49
50     entities = perform_ner(article["rel_abs"])
51     institutions = ner_institutions([author["author_inst"] for author in article["rel_authors"]])
52
53     article["entities"] = entities
54     article["institutions"] = institutions
55
56 def ner_institutions(institutions):
57
58     author_inst_str = ", ".join(institutions)
59
60     doc = nlp(author_inst_str)
61
62     ents = defaultdict(set)
63     for e in doc.ents:
64         ents[e.label_].add(e.text)
65
66     return dict(ents)
67

```

En esta sección se muestran los métodos encargados de parsear los json.

```

69 ESENDPOINT = os.getenv('ESENDPOINT')
70 ESPASSWORD = os.getenv('ESPASSWORD')
71 ESINDEX = os.getenv('ESINDEX')
72 AUGMENTEDINDEX = os.getenv('AUGMENTEDINDEX')
73
74 RABBIT_MQ=os.getenv('RABBITMQ')
75 RABBIT_MQ_PASSWORD=os.getenv('RABBITPASS')
76 SPACY_QUEUE=os.getenv('SPACY_QUEUE')
77
78 credentials = pika.PlainCredentials('user', RABBIT_MQ_PASSWORD)
79 parameters = pika.ConnectionParameters(host=RABBIT_MQ, credentials=credentials)
80 connection = pika.BlockingConnection(parameters)
81 channel = connection.channel()
82 channel.queue_declare(queue=SPACY_QUEUE, durable = True)
83 channel.basic_consume(queue=SPACY_QUEUE, on_message_callback=callback, auto_ack=True)
84
85 # Establish the Elasticsearch connection
86 es = Elasticsearch("http://"+ESENDPOINT+":9200", basic_auth=("elastic", ESPASSWORD), verify_certs=False)
87
88 # Cargar el modelo de Spacy para NER
89 nlp = spacy.load("en_core_web_sm")
90 print(ESINDEX)
91 channel.start_consuming()

```

Finalmente, se define la configuración y variables de ambiente para la conexión a la cola de RabbitMQ y elasticsearch.

Controller

Este componente cumple con la función de revisar continuamente el índice. En el momento que detecta este documento, hace un llamado al API de bioRxiv, capturando el valor llamado messages.total. Con esto el controller genera varios splits, los mismos son una porción de los articulos que se deben descargar. Y por cada split publica un mensaje en RabbitMQ a la cola dada por la variable de entorno CRAWLER_QUEUE.

Implementación Controller

A continuación, se muestra la implementación del componente Controller.

```

1  import requests
2  import pika
3  import os
4  import json
5  import math
6  import time
7  from elasticsearch import Elasticsearch
8  from elasticsearch import TransportError
9
10  def get_biorxiv_data():
11      base_url = 'https://api.biorxiv.org'
12      endpoint = '/covid19/30'
13      headers = {'Content-Type': 'application/json'}
14
15      response = requests.get(base_url + endpoint, headers=headers)
16
17      if response.status_code == 200:
18          return response.json()
19      else:
20          print(f"Error: {response.status_code} - {response.text}")
21          return None
22

```

En la figura anterior se encuentra la implementación del método `get_biorxiv_data`. El cual se encarga de hacer un request al API de bioRxiv y retornando el response en caso de recibir un response status code igual 200. Este método usa la librería de [Requests](#). Está permite enviar solicitudes de HTTP a servicios, como el API.

```

23  RABBIT_MQ=os.getenv('RABBITMQ')
24  RABBIT_MQ_PASSWORD=os.getenv('RABBITPASS')
25  CRAWLER_QUEUE=os.getenv('CRAWLER_QUEUE')
26
27  ESENDPOINT=os.getenv('ESENDPOINT')
28  ESPASSWORD=os.getenv('ESPASSWORD')
29  ESINDEX=os.getenv('ESINDEX')
30
31  credentials = pika.PlainCredentials('user', RABBIT_MQ_PASSWORD)
32  parameters = pika.ConnectionParameters(host=RABBIT_MQ, credentials=credentials)
33  connection = pika.BlockingConnection(parameters)
34  channel = connection.channel()
35  channel.queue_declare(queue=CRAWLER_QUEUE, durable = True)
36
37  data = {
38      "jobId": "1234",
39      "pageSize": "100",
40      "sleep": "2000"
41  }
42  jsonexample = json.dumps(data)
43
44  client = Elasticsearch("http://"+ESENDPOINT+":9200", basic_auth=("elastic", ESPASSWORD), verify_certs=False)
45

```

Posteriormente, se define las variables de entorno necesarias para establecer las conexiones tanto de elasticsearch como de RabbitMQ. Además, se inicializa la conexión al Elasticsearch.

```

46  index_settings = {
47      "mappings": {
48          "properties": {
49              "jobId": {
50                  "type": "keyword"
51              },
52              "pageSize": {
53                  "type": "integer"
54              },
55              "sleep": {
56                  "type": "integer"
57              }
58          }
59      }
60  }
61
62  try:
63      client.indices.create(index=ESINDEX)
64      print(f"Index '{ESINDEX}' created successfully.")
65  except Exception as e:
66      print(f"Index '{ESINDEX}' already exists.")
67
68
69

```

En esta sección se [crea el índice de jobs](#) para el cliente de elasticsearch, tal como se indica en el código.

```

70  indexes = []
71  while True:
72      print("Checking...")
73      # Revisa a que recibe el request del kibana service.
74      try:
75          response = client.search(index=ESINDEX, query = {"match_all": {}})
76      except Exception as e:
77          print("Error:", e)
78          time.sleep(5)
79          continue
80
81      if len(response['hits']['hits']) != 0:
82          print("Found...")
83          for hit in response['hits']['hits']:
84              if hit['_source']['jobId'] in indexes:
85                  continue
86              jsonread = hit['_source']
87              pageSize = int(jsonread["pageSize"]) # extrae el page size del índice
88              apiData = get_biorxiv_data()
89              total = int(apiData["messages"][0]["total"])
90              print(total)
91              print(jsonread)
92
93              splits = math.ceil(total / pageSize)
94              for split in range(splits):
95                  jsonread["splitNumber"] = split
96                  msg = json.dumps(jsonread)
97                  channel.basic_publish(exchange='', routing_key=CRAWLER_QUEUE, body=msg)
98                  indexes.append(jsonread['_source']['jobId'])
99              time.sleep(5)
100
101  connection.close()

```


Finalmente, un ciclo donde se va a procesar el valor de `messages.total` y se generan los splits para publicarlos en la cola de RabbitMQ.

API Crawler

El API Crawler es el componente encargado de leer los mensajes publicados en el RabbitMQ provenientes del Controller. Estos mensajes los lee de la cola llamada `CRAWLER_QUEUE`. Además, este se encarga de descargar los archivos de API bioRxiv y los almacena en un índice de Elasticsearch llamado `raw`. Y al finalizar publica un mensaje en una cola de RabbitMQ.

Implementación API Crawler

A continuación, se muestra la implementación del componente API Crawler.

```

1  import requests
2  import pika
3  import os
4  import json
5  import time
6  from elasticsearch import Elasticsearch
7
8  # Función que extrae datos desde el API de bioRxiv.
9  # Recibe el número de documento a partir va a descargar los 30 artículos
10 def get_biorxiv_data(offset=None):
11     base_url = 'https://api.biorxiv.org'
12     endpoint = '/covid19/'
13     headers = {'Content-Type': 'application/json'}
14
15     response = requests.get(base_url + endpoint + str(offset), headers=headers)
16     print(base_url + endpoint + str(offset))
17
18     if response.status_code == 200:
19         return response.json()
20     else:
21         print(f"Error: {response.status_code} - {response.text}")
22         return None
23

```

En la figura anterior se encuentra la implementación del método `get_biorxiv_data`. El cual se encarga de hacer un request al API de bioRxiv y retornando el response en caso de recibir un response status code igual a 200. Este utiliza la librería de [Requests](#).

```

26  def callback(ch, method, properties, body):
27      json_object = json.loads(body)
28      print(f"Received {json_object}")
29      jobId = json_object["jobId"]
30      pageSize = int(json_object["pageSize"])
31      splitNumber = int(json_object["splitNumber"])
32
33      articles = []
34      articleNumber = pageSize * splitNumber # se calcula el artículo del cual a partir se va a empezar a descar
35      count = 0
36      # se van descargando la cantidad de artículos en el pageSize
37      while(count < pageSize):
38          data = get_biorxiv_data(articleNumber + count)
39          articlesDownloaded = min(len(data["collection"]), pageSize - count)
40          articles += data["collection"][:articlesDownloaded]
41          count += articlesDownloaded
42
43      # se prepara el documento para enviarlo al SPACY_QUEUE
44      json_object["status"] = 'DOWNLOADED'
45      documentId = jobId + str(splitNumber)
46
47      # Se alista el documento para subirlo al índice en elasticsearch.
48      articlesJson = {"splitId": documentId, "articles" : articles}
49      print(splitNumber, len(articles))
50      resp = client.index(index=ESINDEX, id=documentId, document=json.dumps(articlesJson))
51
52      # se publica el mensaje al SPACY_QUEUE
53      channel.basic_publish(exchange='', routing_key=SPACY_QUEUE, body=json.dumps(json_object))
54      time.sleep(int(json_object["sleep"])/1000)
55

```

Posteriormente, en esta figura se muestra la implementación del método callback. El cual se encarga de leer los mensajes de RabbitMQ provenientes del Controller, hace un request al API de bioRxiv para obtener los artículos y los prepara para publicarlos en la cola SPACY_QUEUE.

```

56  # Configuración de RabbitMQ
57  RABBIT_MQ=os.getenv('RABBITMQ')
58  RABBIT_MQ_PASSWORD=os.getenv('RABBITPASS')
59  CRAWLER_QUEUE=os.getenv('CRAWLER_QUEUE')
60  SPACY_QUEUE=os.getenv('SPACY_QUEUE')
61
62  # Configuración de Elasticsearch
63  ESENDPOINT=os.getenv('ESENDPOINT')
64  ESPASSWORD=os.getenv('ESPASSWORD')
65  ESINDEX=os.getenv('ESINDEX')
66
67  # Conexión con RabbitMQ
68  credentials = pika.PlainCredentials('user', RABBIT_MQ_PASSWORD)
69  parameters = pika.ConnectionParameters(host=RABBIT_MQ, credentials=credentials)
70  connection = pika.BlockingConnection(parameters)
71  channel = connection.channel()
72  channel.queue_declare(queue=CRAWLER_QUEUE, durable = True) # cola que recibe mensajes
73  channel.queue_declare(queue=SPACY_QUEUE, durable = True) # cola que envía mensajes al Spacy Entity Extractor
74  channel.basic_consume(queue=CRAWLER_QUEUE, on_message_callback=callback, auto_ack=True)
75
76  client = Elasticsearch("http://" + ESENDPOINT + ":9200", basic_auth=("elastic", ESPASSWORD), verify_certs=False)
77
78  channel.start_consuming()

```

Finalmente se definen las variables de entorno para la configuración de la conexión con RabbitMQ y Elasticsearch.

SparkSQL

SparkSQL es el componente encargado de leer el índice augmented que se ejecutara de forma manual y aplicará ciertas transformaciones a los datos. Una vez transformados, este lo publicará en un índice llamado documents.

Implementación SparkSQL

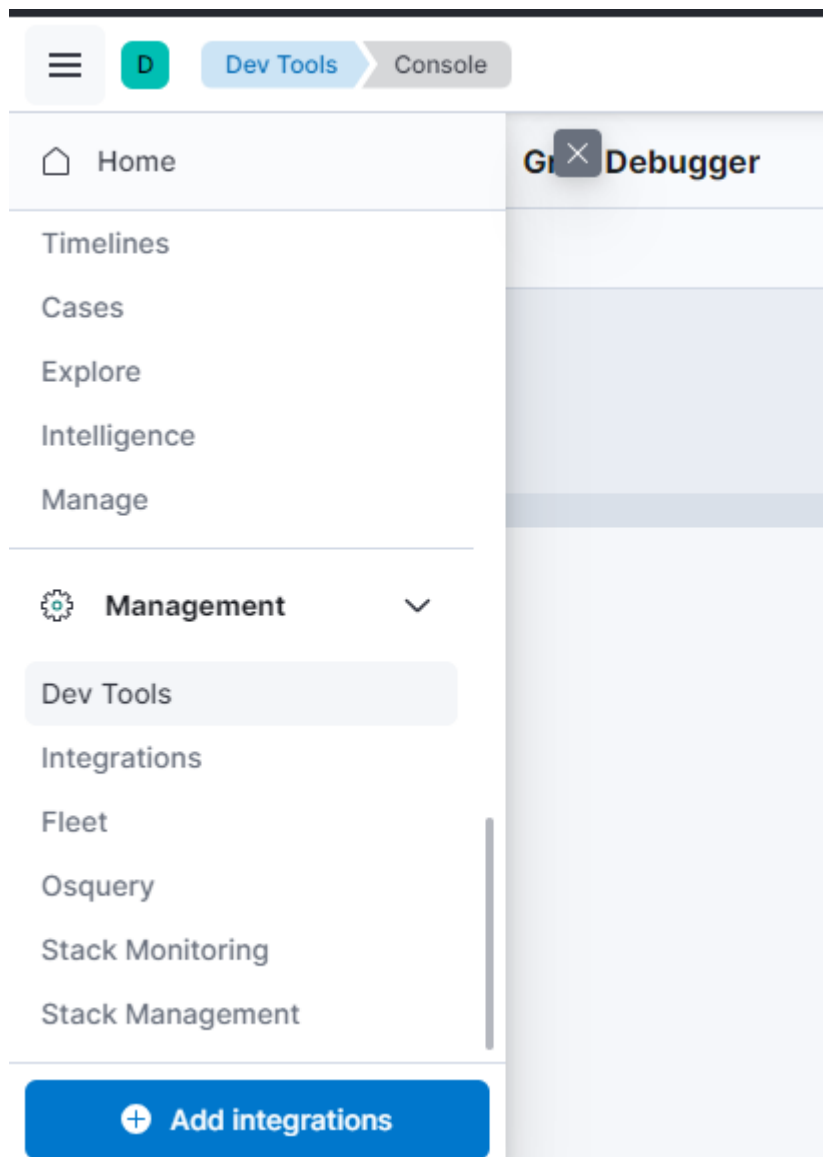
Spark será probado manualmente mediante la consola de Windows, el código de ejecución de este se encuentra [aquí](#). Estos comandos se encuentran debidamente documentados y de igual forma en la Ejecución y Pruebas Realizadas se mostrará el funcionamiento de cada uno ya que se deben de ejecutar manualmente.

Ejecución y Pruebas Realizadas

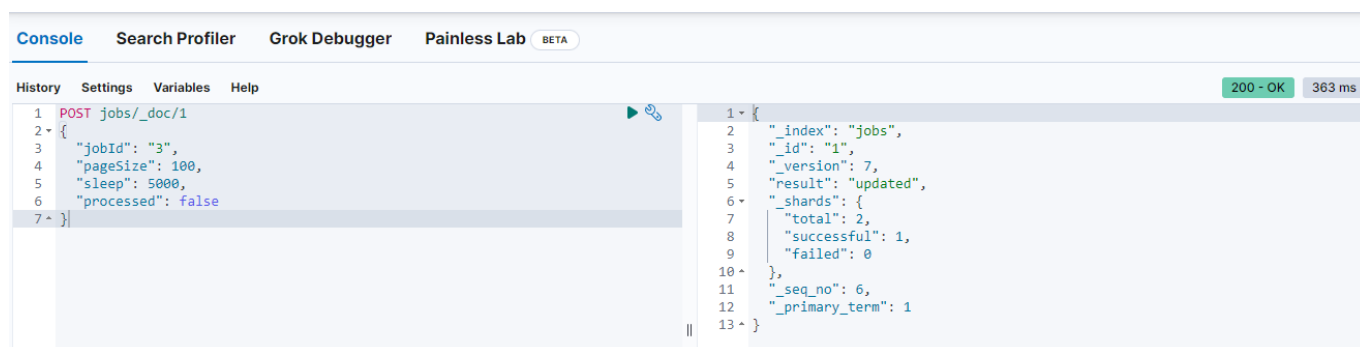
Para iniciar con las pruebas debe de haber sido necesario realizar el proceso de pre-instalación e instalación para el cluster de Kubernetes. Una vez se tiene todo listo se puede iniciar.

El primer paso es acceder a Kibana mediante el NodePort: [Kibana](#) e ingresar las credenciales, el usuario predeterminado es **elastic** y la contraseña se puede acceder por medio de Lens en el apartado de *Config* en el apartado de *Secrets* bajo el nombre de **ic4302-es-elastic-user**. Es importante recordar hacer click sobre el ojo para decodificar la contraseña. Ya decodificada se puede copiar y pegar en Kibana.

Una vez se ingresa a Kibana se abre el menú y se abre el apartado de *Management* bajo el nombre de *Dev Tools* para acceder a la consola de Kibana.



En la consola se ejecutará el siguiente comando el cual en el índice de jobs ingresa un nuevo job con el formato especificado:



Una vez se ingrese el nuevo job, el controller verificará este índice de *jobs* y se dará cuenta que debe de enviar los splits con el mensaje a una cola de RabbitMQ. Cuando el Controller ya lo leyó, actualiza el documento y cambia el campo de *"processed"* a true. De esta manera, no va volver a leer los documentos ya procesados. El crawler entonces empieza a consumir estos mensajes y empieza a extraer la información cruda del BioRxiv API y la guarda en un índice llamado raw, esto se puede verificar con el siguiente comando:

The screenshot shows a REST client interface with a 'Console' tab. A GET request to `raw/_search` is shown with a query object containing `"match_all": {}`. The response is a JSON object with the following structure:

```

{
  "articles": [
    {
      "rel_doi": "10.1101/2022.02.28.22271604",
      "rel_title": "Medium-term impacts of the waves of the COVID-19 epidemic on treatments for non-COVID-19 patients in intensive care units: a retrospective cohort study in Japan",
      "rel_date": "2022-02-28",
      "rel_site": "medRxiv",
      "rel_link": "https://medrxiv.org/cgi/content/short/2022.02.28.22271604",
      "rel_abs": "BackgroundMaintaining critical care for non-Coronavirus-disease-2019 (non-COVID-19) patients is a key pillar of tackling the impact of the COVID-19 pandemic. This study aimed to reveal the medium-term impacts of the COVID-19 epidemic on case volumes and quality of intensive care for critically ill non-COVID-19 patients."
    }
  ]
}

```

Una vez que se van extrayendo los datos, el crawler va publicando mensajes en otra cola llamada `SPACY_QUEUE` para que el Spacy los detecte y empiece con el Named Entity Recognition y guarde las nuevas entidades en un índice que se llama *augmented*, esto se puede verificar con el siguiente comando:

The screenshot shows a REST client interface with a 'Console' tab. A GET request to `augmented/_search` is shown with a query object containing `"match_all": {}`. The response is a JSON object with the following structure:

```

{
  "max_score": 1,
  "hits": [
    {
      "_index": "augmented",
      "_id": "20",
      "_score": 1,
      "_ignored": [
        "articles.rel_abs.keyword"
      ],
      "_source": {
        "splitId": "20",
        "articles": [
          {
            "rel_doi": "10.1101/2023.08.17.553661",
            "rel_title": "Purification, crystallization, and preliminary structural analysis of multivalent immunogenic effector protein-anchored SARS-CoV-2 RBD",
            "rel_date": "2023-08-18",
            "rel_site": "bioRxiv",
            "rel_link": "https://biorxiv.org/cgi/content/short/2023.08.17.553661",
            "rel_abs": "The continuous spread of highly transmissible variants of concern and the potential diminished

```

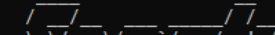
Finalmente como último proceso de verificación se ejecutará el código de SparkSQL con Scala. Como primer paso se debe abrir la consola de Windows y navegar hasta donde se guardó el archivo extraído de la pre-instalación. Una vez dentro de la carpeta, se ingresará a la carpeta de bin y se ejecutará el siguiente comando: `spark-shell` Una vez inicie Spark, se verá de esta forma:

The screenshot shows a Windows command prompt window with the following text:

```

C:\Users\joseg\Desktop\Apps\SparkSQL\spark-2.4.8-bin-hadoop2.7\bin>

```

```
Welcome to
 version 2.4.8

Using Scala version 2.11.12 (Java HotSpot(TM) 64-Bit Server VM, Java 20.0.2)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

```
"query": {
  "match_all": {}
}
```

Códigos Scala

```
// se hacen todos los imports
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.SparkSession._
import org.elasticsearch.spark.sql
import org.elasticsearch.spark.sql._
import org.elasticsearch.spark._
import org.apache.spark.sql.functions
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types.StringType
import org.apache.spark.sql.expressions.UserDefinedFunction
import org.apache.spark.sql.DataFrame
```

La imagen anterior muestra los imports necesarios para ejecutar las pruebas.

```
// Se detiene el contexto y la session actuales para definirlos manualmente.
sc.stop()
spark.stop()

// Se establece la configuración para conectar con Elasticsearch
val conf = new SparkConf()
conf.set("es.index.auto.create", "true")
conf.set("es.nodes", "http://localhost:9200/")
conf.set("es.net.http.auth.user", "elastic")
// cambiar contraseña cada vez que se ejecuta el cluster (secrets de Lens)
conf.set("es.net.http.auth.pass", "YBoD65PpGsG4104T7TD3QT05")
conf.set("es.port", "9200")
conf.set("es.nodes.wan.only", "true")

// Se crea el nuevo contexto y session
val sc = new SparkContext(conf) // El contexto es el punto de entrada al cluster de Spark, lo cual permite manejar

// Un session es un nivel de abstracción que permite trabajar con data frames y datasets.
val spark = SparkSession.builder.config(sc.getConf()).getOrCreate()

// Permite tener una interfaz para trabajar con datos estructurados mediante el uso de SparkSQL.
val sqlcontext = new org.apache.spark.sql.SQLContext(sc)
```

En esta imagen se detiene el contexto inicial y se crea uno propio, seguidamente se configura la conexión a elasticsearch con la credenciales correctas.

```
// Crea funciones que transforman los datos.
val categoryTransform: String => String = category => category.toLowerCase.capitalize

val categoryUDF: UserDefinedFunction = udf(categoryTransform)

val dateTransform: String => String = date => {
  val originalFormat = new java.text.SimpleDateFormat("yyyy-MM-dd")
  val targetFormat = new java.text.SimpleDateFormat("dd/MM/yyyy")
  targetFormat.format(originalFormat.parse(date))
}

val dateUDF: UserDefinedFunction = udf(dateTransform)

val nameTransform: String => String = fullName => {
  if (fullName == null || fullName == ""){
    "No author"
  }
  val parts = fullName.split(" ")
  if (parts.length >= 2) {
    val lastName = parts.last
    val firstName = parts.dropRight(1).mkString(" ")
    s"$lastName, $firstName"
  } else {
    fullName
  }
}
```

```
val nameUDF: UserDefinedFunction = udf(nameTransform)

val componentsTransform: String => String = component => {
  component.replace("\\", "")
}

val componentsUDF: UserDefinedFunction = udf(componentsTransform)

val augmentedDF = spark.read.format("org.elasticsearch.spark.sql").option("es.read.field.as.array.include", "arti")
augmentedDF.printSchema()
augmentedDF.show(false)
```



```
// Crea funciones que transforman los datos.

augmentedDF.createOrReplaceTempView("temp_view")

val result = spark.sql("""
SELECT
  article.rel_title AS title,
  article.category AS category,
  article.rel_date AS rel_date,
  author.author_name AS author_name,
  author.author_inst AS author_inst,
  author.institutions AS components
FROM
  temp_view
  LATERAL VIEW explode(articles) AS article
  LATERAL VIEW explode(article.rel_authors) AS author
""")

result.show(50)

val transformedDF = result.withColumn("title", col("title")).withColumn("category", categoryUDF(col("category")))

transformedDF.show(50)

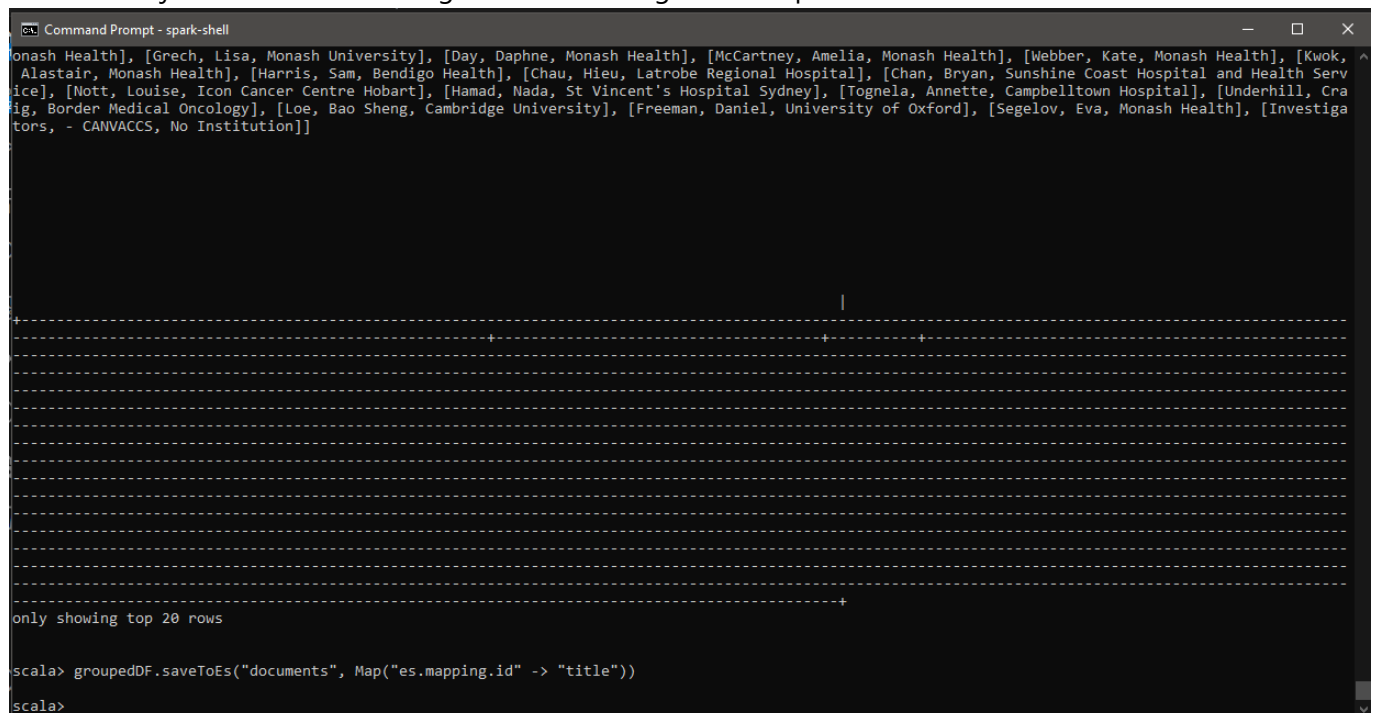
val groupedDF = transformedDF.groupBy("title").agg(first("category").alias("category"), first("rel_date").alias("rel_date"))
groupedDF.show(50)
groupedDF.saveToEs("documents", Map("es.mapping.id" -> "title"))
```

Finalmente, se crean la funciones transforman los datos, por ejemplo:

- El author_name, deberá convertirse en formato Apellido, Nombre.
- El author_inst deberá separarse en sus componentes
- El category debe convertir cada primera letra de palabra en mayúscula y remover espacios a los lados.
- El rel_date debe convertirse al formato dd/mm/yyyy. Y luego se suben a un nuevo índice en elasticsearch con el nombre de documents.

Ejecución

Cuando se ejecute se deberá ver algo similar a las siguientes capturas:



```
Command Prompt - spark-shell
Monash Health], [Grech, Lisa, Monash University], [Day, Daphne, Monash Health], [McCartney, Amelia, Monash Health], [Webber, Kate, Monash Health], [Kwok, Alastair, Monash Health], [Harris, Sam, Bendigo Health], [Chau, Hieu, Latrobe Regional Hospital], [Chan, Bryan, Sunshine Coast Hospital and Health Service], [Mott, Louise, Icon Cancer Centre Hobart], [Hamad, Nada, St Vincent's Hospital Sydney], [Tognola, Annette, Campbelltown Hospital], [Underhill, Craig, Border Medical Oncology], [Loe, Bao Sheng, Cambridge University], [Freeman, Daniel, University of Oxford], [Segelov, Eva, Monash Health], [Investigators, - CANVACCS, No Institution]]

+-----+-----+-----+-----+-----+-----+
|title|category|rel_date|author_name|author_inst|components|
+-----+-----+-----+-----+-----+-----+
|Monash Health|Monash Health|2019-01-01|Grech, Lisa|Monash University|Monash Health|
|Monash Health|Monash Health|2019-01-01|Day, Daphne|Monash Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|McCartney, Amelia|Monash Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|Webber, Kate|Monash Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|Kwok, Alastair|Monash Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|Harris, Sam|Bendigo Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|Chau, Hieu|Latrobe Regional Hospital|Monash Health|
|Monash Health|Monash Health|2019-01-01|Chan, Bryan|Sunshine Coast Hospital and Health Service|Monash Health|
|Monash Health|Monash Health|2019-01-01|Mott, Louise|Icon Cancer Centre Hobart|Monash Health|
|Monash Health|Monash Health|2019-01-01|Hamad, Nada|St Vincent's Hospital Sydney|Monash Health|
|Monash Health|Monash Health|2019-01-01|Tognola, Annette|Campbelltown Hospital|Monash Health|
|Monash Health|Monash Health|2019-01-01|Underhill, Craig|Border Medical Oncology|Monash Health|
|Monash Health|Monash Health|2019-01-01|Loe, Bao Sheng|Cambridge University|Monash Health|
|Monash Health|Monash Health|2019-01-01|Freeman, Daniel|University of Oxford|Monash Health|
|Monash Health|Monash Health|2019-01-01|Segelov, Eva|Monash Health|Monash Health|
|Monash Health|Monash Health|2019-01-01|Investigators|CANVACCS|Monash Health|
|Monash Health|Monash Health|2019-01-01|No Institution|No Institution|Monash Health|

only showing top 20 rows

scala> groupedDF.saveToEs("documents", Map("es.mapping.id" -> "title"))
scala>
```

Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Geir Bukholm	Norwegian Institu...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Anja Brathen Kri...	Norwegian Institu...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Kenth Eng?-Monsen	Smart Innovation ...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Solveig Engebretsen	Norwegian Computi...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	David Swanson	The University of...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Alfonso Diz-Lois...	Norwegian Institu...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Jonas Christoffe...	Norwegian Institu...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Arnoldo Frigessi	University of Osl...
Modeling geograph...	infectious diseases	2023-08-21 00:00:00	Birgitte Freiesl...	Norwegian Institu...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Lesly Ibarguen Go...	Health Research I...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Sandra Heller	Institute of Mole...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Marta L DeDiego	Centro Nacional d...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Dario Lopez-Garcia	Department of Mol...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Alba M Gomez-Valero	Health Research I...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Thomas FE Barth	Department of Pat...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Patricia Gallego	Hospital Son Espases
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Israel Fernandez...	Stroke Pharmacoge...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Sayoa Alzate-Pinol	Stroke Pharmacoge...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Catalina Crespi	Health Research I...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Julieth A Mena-Gu...	Health Research I...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Maria Eugenia Cis...	Health Research I...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Alejandro P Ugalde	Departamento de B...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Gabriel Bretones	Universidad de Ov...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Charlotte Steenblock	Department of Int...
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Alexander Kleger	Ulm University
Host factor PLAC8...	cell biology	2023-08-21 00:00:00	Carles Barcelo	Health Research I...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Irene Unterman	Department of Dev...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Dana Avrahami	Department of End...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Efrat Katsman	Department of Dev...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Timothy J Triche Jr.	Center for Epigen...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Benjamin Glaser	Department of End...
Multi-cell type d...	bioinformatics	2023-08-21 00:00:00	Benjamin P Berman	Department of Dev...
Differences in OL...	biophysics	2023-08-20 00:00:00	Julia A. Townsend	University of Ari...
Differences in OL...	biophysics	2023-08-20 00:00:00	Oluwaseun Fapohunda	University of Ari...
Differences in OL...	biophysics	2023-08-20 00:00:00	Zhihan Wang	University of Ari...
Differences in OL...	biophysics	2023-08-20 00:00:00	Hieu Pham	University of Ari...
Differences in OL...	biophysics	2023-08-20 00:00:00	Michael T. Taylor	University of Ari...

Console
Search Profiler
Grok Debugger
Painless Lab
BETA

History	Settings	Variables	Help																																								
<pre> 11 * { 12 "match_all": {} 13 ^ } 14 * } 15 16 GET augmented/_search 17 * { 18 "query": { 19 "match_all": {} 20 ^ } 21 * } 22 23 GET _cat/indices 24 25 GET raw/_search 26 * { 27 "query": { 28 "match_all": {} 29 ^ }</pre>	200 - OK 251 ms <table border="1" style="width: 100%; border-collapse: collapse; font-family: monospace;"> <tr> <td style="padding: 2px 5px;">1</td> <td style="padding: 2px 5px;">yellow open documents</td> <td style="padding: 2px 5px;">qzUmK0byRGac7Zw1Q_mzQA</td> <td style="padding: 2px 5px;">1 1</td> <td style="padding: 2px 5px;">3799</td> <td style="padding: 2px 5px;">3699</td> <td style="padding: 2px 5px;">15.1mb</td> <td style="padding: 2px 5px;">15.1mb</td> </tr> <tr> <td style="padding: 2px 5px;">2</td> <td style="padding: 2px 5px;">yellow open jobs</td> <td style="padding: 2px 5px;">jwxHMaSvEQumG2KdjmjZU1w</td> <td style="padding: 2px 5px;">1 1</td> <td style="padding: 2px 5px;">1</td> <td style="padding: 2px 5px;">3</td> <td style="padding: 2px 5px;">25.6kb</td> <td style="padding: 2px 5px;">25.6kb</td> </tr> <tr> <td style="padding: 2px 5px;">3</td> <td style="padding: 2px 5px;">yellow open raw</td> <td style="padding: 2px 5px;">CEJYyaVnWQZubSK_Bqn1Yg</td> <td style="padding: 2px 5px;">1 1</td> <td style="padding: 2px 5px;">136</td> <td style="padding: 2px 5px;">0</td> <td style="padding: 2px 5px;">46.4mb</td> <td style="padding: 2px 5px;">46.4mb</td> </tr> <tr> <td style="padding: 2px 5px;">4</td> <td style="padding: 2px 5px;">yellow open augmented</td> <td style="padding: 2px 5px;">1JHHEQtQSe6OXjOGURrf8A</td> <td style="padding: 2px 5px;">1 1</td> <td style="padding: 2px 5px;">39</td> <td style="padding: 2px 5px;">0</td> <td style="padding: 2px 5px;">18.1mb</td> <td style="padding: 2px 5px;">18.1mb</td> </tr> <tr> <td style="padding: 2px 5px;">5</td> <td colspan="7"></td> </tr> </table>			1	yellow open documents	qzUmK0byRGac7Zw1Q_mzQA	1 1	3799	3699	15.1mb	15.1mb	2	yellow open jobs	jwxHMaSvEQumG2KdjmjZU1w	1 1	1	3	25.6kb	25.6kb	3	yellow open raw	CEJYyaVnWQZubSK_Bqn1Yg	1 1	136	0	46.4mb	46.4mb	4	yellow open augmented	1JHHEQtQSe6OXjOGURrf8A	1 1	39	0	18.1mb	18.1mb	5							
1	yellow open documents	qzUmK0byRGac7Zw1Q_mzQA	1 1	3799	3699	15.1mb	15.1mb																																				
2	yellow open jobs	jwxHMaSvEQumG2KdjmjZU1w	1 1	1	3	25.6kb	25.6kb																																				
3	yellow open raw	CEJYyaVnWQZubSK_Bqn1Yg	1 1	136	0	46.4mb	46.4mb																																				
4	yellow open augmented	1JHHEQtQSe6OXjOGURrf8A	1 1	39	0	18.1mb	18.1mb																																				
5																																											

elastic

Find apps, content, and more.

D

Dev Tools

Console

Help us improve the Elastic Stack

To learn about how usage data helps us manage and improve our products and services, see our [Privacy Statement](#). To stop collection, [disable usage data](#) here.

Dismiss

Console

Search Profiler

Grok Debugger

Painless Lab

BETA

History

Settings

Variables

Help

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

```

GET _cat/indices
GET raw/_search
{
  "query": {
    "match_all": {}
  }
}
GET documents/_search
{
  "query": {
    "match_all": {}
  }
}

```

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

```

"rel_date": "22/06/2023",
"authors": [
  {
    "author_name": "Roquencourt, Camille",
    "author_inst": "Hopital Foch, Exhalomics, Suresnes, France"
  },
  {
    "author_name": "Lamy, Elodie",
    "author_inst": "Universite Paris-Saclay, UVSQ, INSERM U1173
, Infection et inflammation, Departement de
Biotechnologie de la Sante, Montigny le Bretonneux,
France"
  },
  {
    "author_name": "Bardin, Emmanuelle",
    "author_inst": "Institut Necker-Enfants Malades, Paris,
France"
  },
  {

```

22 ms

45 ms

Respecto a las pruebas unitarias, debido al poco código generado durante el desarrollo del proyecto, únicamente se logran identificar 2 componentes a los cuales se les puede realizar un unit test, los cuales serán mencionados más adelante. Esto bajo la premisa de probar únicamente código generado para la implementación del "motor de búsqueda" de artículos científicos, omitiendo las funciones que generen conexiones con los distintos componentes del sistema, tales como RabbitMQ, Elasticsearch y Spacy.

Controller

Referente al controller, este cuenta únicamente con un método, `get_biorxiv_data`, el cual se encarga de hacer un request al API de bioRxiv y retornar un response. El response retornado por `get_biorxiv_data` puede ser null o bien un json, esto depende del response status code proveniente del API. Debido a que no podemos controlar el comportamiento del API de bioRxiv, no se puede generar casos de prueba sobre los posibles errores y se procede únicamente a validar el resultado proveniente del método anteriormente mencionado.

A continuación, se muestra el código del unit test generado para el método del controller.

```

1  import unittest
2  import controller
3
4  class TestBiorxiv(unittest.TestCase):
5
6      response = None
7
8      def setUp(self):
9          response = controller.get_biorxiv_data()
10
11      def testNoneResponse(self):
12          self.assertIsNone(response, "The response is not supposed to be None")
13
14      def tearDown(self):
15          response = None
16
17  if __name__ == '__main__':
18      unittest.main()

```

El resultado de esta prueba es positivo ya que funciona correctamente al ejecutar el sistema. En caso de recibir un status code diferente a 200 este indica el error.

Crawler

Caso similar al controller, el crawler únicamente cuenta con 2 métodos, de los cuales solo el método `get_biorxiv_data` es sujeto para un unit test. Este se encarga de hacer un request al API de bioRxiv y recibir un response con los artículos científicos. El unit test para el mismo se presenta en la siguiente figura.

```

1  import unittest
2  import crawler
3
4  class TestBiorxiv(unittest.TestCase):
5
6      response = None
7
8      def setUp(self):
9          response = crawler.get_biorxiv_data()
10
11      def testNoneResponse(self):
12          self.assertIsNone(response, "The response is not supposed to be None")
13
14      def tearDown(self):
15          response = None
16
17  if __name__ == '__main__':
18      unittest.main()

```

El resultado de esta prueba es positivo ya que funciona correctamente al ejecutar el sistema. En caso de recibir un status code diferente a 200 este indica el error.

Recomendaciones y Conclusiones

A continuación, se presentan una serie de recomendaciones con el fin de un desarrollo más eficiente de los componentes que forman parte de este sistema. Adicionalmente, se muestran una serie de conclusiones al momento de finalizar el proyecto.

Recomendaciones

1. Mejorar el trabajo en equipo.
2. Una buena distribución del trabajo.
3. Estudiar a profundidad cada una de las tecnologías a trabajar.
4. Invertir tiempo en aprender Kubernetes.
5. Establecer consultas periódicas y puntuales con el profesor a cargo.
6. Establecer entregas periódicas.
7. Buena comunicacion de equipo.
8. Adecuada organizacion del GIT(repositorio).
9. Mejorar las prácticas de programacion.
10. Realizar reuniones grupales periódicas donde todos los integrantes participen.
11. Incorporar estrategias de métodos ágiles como SCRUM.
12. Guardar las fuentes de donde se extrajo el código.
13. Escribir archivos de automatización por medio de archivos .sh o Makefile. Estos pueden ser usados para instalar helm charts automáticamente o para subir imágenes a Docker Hub.
14. Hacer imágenes de Docker para el desarrollo que utilicen un bind mount para no tener que recompilar la imagen.

Conclusiones

1. Para el manejo de datos semi-estructurados no es posible utilizar una base de datos SQL de forma óptima.

2. Elasticsearch es muy eficiente con grandes volúmenes de datos.
3. La configuración establecida entre elasticsearch y python fue realizada con facilidad, por lo que las librerías utilizadas son intuitivas y fáciles de configurar.
4. Los mensajes de RabbitMQ funcionan de manera muy eficiente y rápida para el programa desarrollado. Resaltando algunas de las características competitivas del message broker.
5. Docker es realmente intuitivo una vez comprendida la teoría. Obteniendo así una eficaz herramienta para el desarrollo de software.
6. Kubernetes es una herramienta complicada de utilizar. Sin embargo, agrega gran beneficio en temas de automatización, escalado y administración de contenedores.
7. A nivel de cluster puede llegar a ser compleja la búsqueda y resolución de errores.
8. Se genera un aprendizaje básico en cuanto a la arquitectura de microservicios.
9. Es necesario leer la documentación de los helm charts para entender el funcionamiento y parámetros requeridos para el correcto funcionamiento de los mismos.
10. Se comprende de forma básica los servicios de extracción de entidades.

Referencias

1. unittest — Unit Testing framework. (s. f.). Python documentation.
<https://docs.python.org/3/library/unittest.html>
2. unittest — Unit Testing framework. (s. f.). Python documentation.
<https://docs.python.org/3/library/unittest.html#unittest.TestCase.assertEqual>